Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

Abstract

Large language models (LLMs) repair buggy submissions to programming assignments well, but they are deployed *one query per submission*: every buggy program triggers a fresh model call, so cost, latency, and external dependence scale with enrollment and recur every term, while the specification-level reasoning being purchased is identical across a class — and caching or cheaper models shrink only the constant, not the structure. We propose CoEP, which moves the LLM from the marginal-cost to the fixed-cost column: *spend the model once per assignment, not once per submission*. A cheapest-tier LLM agent reads the assignment and a clustered sample of worked (buggy → fixed) pairs and authors the assignment-specific layer of a local repair tool — AST rewrite rules named after the misconceptions they fix — atop a shared symbolic substrate of generic edits and reference-guided minimal-hunk transplant from the class's own correct solutions. Every candidate patch is validated against the instructor's tests before it is kept; the tool can never self-report success, and submissions it abstains on escalate to a conventional per-submission call, so coverage is never sacrificed. On the standard benchmark of this literature, the synthesized tool repairs *more* submissions with *smaller* patches than the strongest hand-built symbolic tool, on that tool's own corpus and metric — at a few cents of one-off build cost per assignment, zero marginal LLM cost for everything repaired locally, about a second per repair on a CPU, offline and deterministic. A price-invariant break-even analysis shows when amortization wins and why cheaper models and caching do not change the answer, and the artifact yields what a per-query API cannot: reproducible minimal patches and a per-class misconception profile.

Introduction

Timely, individualized feedback is the scarcest resource in programming education: introductory courses enroll hundreds of students on campus and tens of thousands online, each producing buggy submissions no instructor can examine one by one. Automated program repair (APR) has therefore long been proposed as a feedback engine for programming assignments: given a buggy submission and the assignment's test suite, produce a small patch that makes it correct while preserving the student's own approach (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022).

Three generations of systems have pursued this goal. The first established the setting: test-driven repair of introductory assignments, with the requirement that a patch stay close to the student's program, since a correct but unrelated rewrite passes the tests and teaches nothing (Yi et al. 2017). The second exploited the structure of a *class*, mining the assignment's own solution space: CLARA clusters correct submissions, Sarfgen searches aligned examples, Refactory refactors correct solutions into a per-problem search space, and Verifix aligns the student's program to a reference under verified equivalence (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022). These repair each submission cheaply, at two prices: an assignment must first accumulate a large pool of correct solutions, and repair proceeds by matching the student's control flow against one of them, so a submission whose structure diverges from every reference is out of reach (Orvalho, Janota, and Manquinho 2025). Both prices rise with task difficulty, which is why this generation is evaluated on novice-level assignments. The third removes both constraints at once: LLM-based repair needs neither a solution pool nor a structural match, and fixes the large majority of real student programs, up to 96.5% of buggy submissions in PyDex (Zhang et al. 2024), at modest per-bug prices (Xia and Zhang 2024, 2022; Liu et al. 2024; Yang et al. 2024).

What the third generation fixed in coverage, however, it broke in *cost structure*. Its deployment default is one model query per buggy submission, so spending scales linearly with enrollment and restarts every term — while the queries themselves are massively redundant. Assignment repair is a *many-instances, one-specification* regime: one handout is attempted by hundreds or thousands of students whose failures cluster into a handful of recurring misconceptions (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019); on the corpus we study, 77% of a repair prompt is byte-identical across submissions to the same assignment. Paying a model to rediscover the same off-by-one for the three-hundredth time is not intelligence at work; it is a billing artifact. The standard remedies do not change this shape: cheaper models and provider caching (Anthropic 2025; OpenAI 2024) shrink the *constant*, retrieval enriches the query (Dai et al. 2026), but

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

every submission still triggers a frozen-model round trip, with the attendant marginal cost, latency, rate limits, nondeterminism, and shipping of student code to an external service. The question worth asking is not how much a call costs, but *where in the pipeline the LLM is invoked*.

Our answer moves the LLM from the marginal-cost column to the fixed-cost column: **spend the LLM once per assignment, not once per submission**. Agent research has shown that LLMs can author reusable code once and amortize it across many uses (Cai et al. 2024; Wang et al. 2024); classical program synthesis has even learned per-assignment fix rules from examples (Rolim et al. 2017). We instantiate this idea for assignment repair, under the discipline the setting demands — every patch validated against the instructor's tests, never trusted from generation. In CoEP (Cost-Effective Program repair), a *cheapest-tier* LLM agent ($0.004–$0.05 of inference per assignment) reads the specification, tests, and a clustered sample of worked (buggy → fixed) pairs, and authors the assignment-specific layer of a local repair tool: AST rewrite rules named after the misconceptions they fix. That layer runs on a shared symbolic substrate — a generic small-edit library and a reference-guided *minimal-hunk transplant* engine that adopts the smallest differing hunks from the structurally nearest correct solution, in the lineage of reference-reuse repair (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019). Every candidate is executed against the visible tests before it is kept, and the builder loop refines the module against harness feedback; a tool can never self-report success. The frozen tool then repairs submissions locally, in about a second on a CPU, at *zero* marginal LLM cost, and the few it abstains on escalate to a conventional per-submission call, so the hybrid never sacrifices coverage.

The resulting artifact does not merely approximate the hand-built state of the art — it beats it on its own benchmark and its own metric. Against Refactory (Hu et al. 2019), CoEP's local repair covers more submissions at smaller patch size (97.8% vs. 90.8% repair rate, 0.326 vs. 0.40 mean relative patch size over 1,686 real student programs). We are explicit about attribution: the LLM-authored rules produce $\sim$13% of local fixes — the near-minimal ones (RPS $\approx$0.03), each carrying a diagnostic misconception label — while the hand-written transplant engine produces the rest. The claim is therefore not that an LLM wrote a SOTA repairer; it is that a few cents of once-per-assignment LLM spend, composed with a reusable symbolic substrate and a test harness as sole adjudicator, *suffices to exceed* a hand-built system, while removing the per-submission inference entirely for everything repaired locally.

At today's cheapest-tier prices the absolute dollars are small; the structure is not. If building a tool costs C_{build} and it locally resolves a fraction $(1-f)$ of submissions that would each have cost Δ, amortization wins once enrollment exceeds $N^* = C_{\text{build}}/(\Delta(1-f))$ — and when build and per-query spending are priced on the same token schedule, N^* is *invariant to token price*: "just use a cheaper model" shrinks both sides of the ledger and leaves the crossover untouched. Meanwhile the fixed-cost artifact carries properties per-submission inference cannot offer at any price, scoped to the locally repaired fraction: it assembles patches on the AST, so it cannot emit syntactically invalid code and splices minimal spans rather than regenerating files; it is deterministic and reproducible; it runs offline, so student code never leaves the institution and repair latency does not grow with enrollment; and its rule-fired repairs aggregate into a per-class misconception profile an API call never returns.

This paper makes the following contributions:

- **A cost-structure reformulation of assignment repair.** We move the LLM from per-submission marginal cost to per-assignment fixed cost, with a break-even analysis $N^* = C_{\text{build}}/(\Delta(1-f))$ that is price-invariant under a stated pricing assumption — amortization is a structural advantage, not a constant-factor one that cheaper models erase.
- **The CoEP system.** A cheapest-tier builder agent synthesizes misconception-named AST rewrite rules through a generate–validate–refine loop (Madaan et al. 2023; Shinn et al. 2023), atop a shared substrate of generic edit rules and reference-guided minimal-hunk transplant with variable renaming; every patch is independently test-validated, so a tool can never self-report success, and abstentions escalate to a strategy-matched LLM fallback.
- **An empirical result on the baseline's own corpus and metric.** CoEP's local repair exceeds Refactory on both coverage and patch size at $0.004–$0.05 of build cost per assignment, deterministic and offline at 1–2 s per submission, with an honest mechanism attribution ($\sim$13% LLM rules, $\sim$87% transplant) and per-builder analyses across two commodity models.
- **Artifacts.** We release the harness, dataset-import pipeline, prompts, synthesized repairers, and all raw model outputs.[1]

The Cost Anatomy of Per-Submission Repair

Decomposing the per-submission prompt of our PyDex reproduction (and, mutatis mutandis, of ChatRepair-style systems (Zhang et al. 2024; Xia and Zhang 2024)) shows where the money goes. Averaged over the Refactory corpus, a query consumes 439 input and 94 output tokens: a preamble with the assignment description and visible tests (241 tokens, 55%), the buggy-submission section (99 tokens, 23%), which is the *only* part that varies within an assignment, and fixed instructions (99 tokens, 23%). Only the contiguous 55% prefix ahead of the code is even cacheable, and richer prompt styles widen the gap: the PyDex-style few-shot variant re-transmits 1,493 identical tokens per query, 88% of its input.

Two consequences follow. (1) *Caching is a discount, not a fix; here, not even a discount.* Prompt caching bills a shared prefix at $0.1\times$ after a $1.25\times$ write (Anthropic 2025); with no minimum it would cut per-submission cost 23%, but CS1 repair prompts (439 tokens) fall below the provider's 1,024-token cacheable minimum, so even that bound is unreachable; caching does engage on the bloated few-shot prompt (1,705 tokens; $6.5 to 2.9×10^{-3}): the redundancy is real, but provider caching serves it at the wrong granularity.

[1] Anonymized repository submitted as supplementary material.

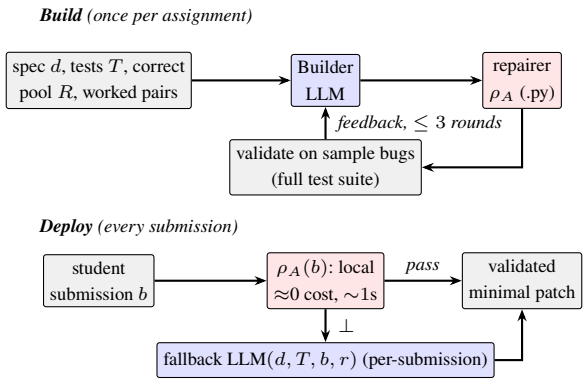


Figure 1: CoEP. LLM effort (blue) is spent once per assignment to synthesize a local repairer (red); at deployment, the large majority of submissions never touch a model.

(2) *The redundancy is exactly the amortizable part.* What is constant per assignment, namely understanding the specification, the tests, and the space of likely mistakes, can be reified *once* into an executable artifact. That is the move CoEP makes, and the rest of the paper quantifies what it buys.

Approach

Problem Formulation

An *assignment* is a triple $A = (d, T, R)$: a natural-language description d, a test suite $T = \{(x_i, y_i)\}$ (function-entry or judge-style stdin/stdout), and the pool R of correct student solutions, which the setting assumes is available at feedback time — as every per-assignment system from CLARA to Refactory assumes (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019). A *repair* of a buggy submission b is a program p that passes all of T. Patch quality is measured, never gated: RPS, the tree-edit distance between the ASTs of b and p normalized by the size of b (computed with Refactory's reference implementation (Hu et al. 2019)), is reported for every repair, since a small patch is feedback on the *student's* program while a rewrite passes tests but teaches nothing (Yi et al. 2017).

Conventional LLM-based APR maps each b to a query $\mathrm{LLM}(d, T, b)$. CoEP instead synthesizes, once per assignment, a *repairer* $\rho_A : b \mapsto p \cup \{\perp\}$, an ordinary module with no model access, and deploys the hybrid: $\mathrm{repair}(b) = \rho_A(b)$ if $\rho_A(b) \neq \perp$ (local, $\approx$0 marginal cost), else $\mathrm{LLM}(d, T, b, r)$ — the conventional call, strengthened with one reference r chosen to match the student's strategy (cluster R on program structure; draw r from the cluster nearest b).

Repairer Synthesis (once per assignment)

Build-set selection (local, no LLM cost). The builder does not see the whole pool; it sees a *chosen* sample. We cluster the buggy pool on a structural view (TF-IDF over normalized-AST paths, identifiers folded) and, walking clusters largest-first, take from each the member with the smallest tree-edit distance to its nearest correct solution — a clean, single-misconception exemplar rather than the medoid's typical-but-tangled program — paired with that nearest correct program. The result is a set of worked (buggy $\rightarrow$ fixed) pairs: frequent misconceptions first, each shown *with* its minimal fix, so a weak builder needs only to generalize a transformation it is shown, not infer it.

Rule authoring. The builder LLM receives d, T, reference solutions (structural-cluster medoids of R), and the worked pairs, and emits the assignment-specific rule module: AST rewrite rules, each named after the misconception it targets and tagged with the failure symptoms it addresses. The contract is strict: locate the edit on the AST, splice it as a source-span replacement (never re-render the module, which reformats untouched lines), emit *candidates* rather than claimed fixes, and never rewrite a whole function — a rule that cannot express a targeted edit must abstain so the submission escalates.

Generate–validate–refine. A single-shot module can contain its own defects, so the harness closes the loop: it loads the module, runs it on the sample bugs, *independently* validates every candidate patch by executing the full suite (the repairer's own claims are never trusted), and, while any sample remains unrepaired, returns the module with per-bug outcomes and the still-failing sources for revision, up to $R_{\max}$=3 builder calls. The best-validated round is kept. Build cost is exactly the token count of these $\leq R_{\max}$ exchanges, measured from the raw strings — \$0.004–\$0.05 per assignment at the cheapest tiers.

Local Repair (every submission)

The frozen tool repairs in three stages, cheapest and smallest-patch first, validating every candidate against the full suite before it is kept:

(1) Named rules + generic search. Rules are *candidate generators*, composed by a symptom-ordered combination search: the submission's failing-test signature ranks rules whose declared symptoms match, then single rules, then pairs, under an explicit candidate budget. A shared generic library (comparison flips, off-by-one range and index shifts, Boolean-operator flips — the inversion of the classical mutation-operator space (Just et al. 2014; Kim et al. 2013; Liu et al. 2019)) is always on, so common slips are covered regardless of what the builder wrote. Every patch that survives carries the ordered list of rule names that produced it — provenance that aggregates into a per-class *misconception profile*.

(2) Reference-guided minimal-hunk transplant. Most student bugs need a structural edit no token-level rule expresses; the correct pool supplies it. We diff b against its structurally nearest correct solutions, rename the reference's variables to the student's (parameters by position, locals by first-appearance order), and adopt the *smallest* set of differing line-hunks that passes the tests, candidates ordered smallest-first and batch-validated, returning the minimum-RPS passer. This is reference reuse in the lineage of CLARA, Sarfgen, and Refactory (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019), at hunk granularity rather than block granularity — which is what keeps patches smaller than the hand-built tool's.

(3) Escalation and self-learning. A submission both stages abstain on takes the fallback $\mathrm{LLM}(d, T, b, r)$, its cost counted against CoEP in every comparison. Each successful fallback fix can be generalized by the LLM into a new named rule, appended only if the augmented tool then repairs that miss on its own (else rolled back) — so the tool accumulates coverage across a term; we report this mechanism as implemented but leave its longitudinal evaluation to future work.

The local path runs on a CPU in $\sim$1–2 s per submission, deterministically and offline; a defective tool degrades to $\perp$ (escalation), never to a wrong "repair", because the grading harness re-validates every patch before a student sees it.

Experimental Setup

We ask three research questions: **RQ1 (Effectiveness & quality).** How do CoEP's synthesized repairers compare to per-submission frontier-LLM repair in repair rate and patch quality? **RQ2 (Cost).** What are the token/dollar costs of both paradigms, when does repairer synthesis break even, and what is the asymptotic saving? **RQ3 (Sources of power).** How much do the synthesized problem-specific rules and the refinement rounds contribute?

Dataset

We evaluate on 11 problems from two real-student corpora (Table 1), 2,970 buggy submissions in all, covering the two execution models an assignment-repair harness must support. **Refactory** (Hu et al. 2019) gives five CS1 assignments by 361 NUS students, the standard corpus of this literature (Ahmed et al. 2022; Zhang et al. 2024; Orvalho, Janota, and Manquinho 2025); their tests call named functions (one assignment needs three), so they run in *function-entry* mode. **Codeforces** gives six competitive-programming problems from the open-r1 release (Penedo et al. 2025), which read stdin/stdout (*judge* mode) and reach beyond CS1. We select the latter by construction: from the *verifiable* subset, we keep problems with $\geq$1 wrong and $\geq$1 accepted Python 3 submission, *complete* official tests (Codeforces truncates tests over 400 characters; we require none were dropped, so the oracle is the full judged suite), and no special-output checker; to span difficulty we take one representative per rating band (Codeforces), the problem with the most wrong submissions.

Oracle fidelity. A weak suite would let a wrong program pass, inflating every arm's repair rate, so we validate the imported oracles rather than trust the labels: running every Codeforces submission against the full official suite, 1 of 1,187 wrong programs passes and 2 of 698 accepted programs fail (0.16%), the three anomalies excluded. The check also surfaced two runner gaps we fixed—file-mode I/O (`input.txt`/`output.txt`) and the fast-I/O idiom (`sys.stdin.buffer`).

Both corpora are plausibly in every current LLM's training data; we use them *deliberately*, since memorization inflates every LLM arm symmetrically (baselines and builder share a model) while leaving the non-LLM tool and generic search untouched, and cannot pass our harness on the builder's behalf. We analyze residual asymmetries in Discussion.

Problem	Band	Rat.	Buggy	Corr.	Tests
Codeforces (judge mode)					
110/A	Newbie	800	440	353	34
131/A	Pupil	1000	560	140	56
1011/B	Apprentice	1200	69	164	31
125/A	Specialist	1400	38	37	22
11/B	Expert	1600	77	2	30
10/C	Master	2000	3	2	40
Refactory (function-entry mode)					
`search`	CS1	–	575	768	11
`unique_day`†	CS1	–	435	291	9
`remove_extras`	CS1	–	308	546	6
`sort_age`	CS1	–	357	419	6
`top_k`	CS1	–	108	418	5
Total			2,970	3,140	

Table 1: The 11 problems. "Buggy" = wrong submissions (repair targets), "Corr." = correct submissions (reference pool). †: requires three functions.

Baselines

We compare four systems along one non-LLM tool and, for each of two LLM models, two per-submission baselines and CoEP. **Refactory** (Hu et al. 2019) is the classical non-LLM per-assignment tool, run in its strongest published configuration (online refactoring, structure mutation, block repair, 100% sampling of correct solutions): for each assignment it receives *all* correct student submissions plus the reference, and its output programs are re-validated under our harness and metrics. **PyDex** (Zhang et al. 2024) is the per-submission LLM paradigm: we reproduce its pipeline — a multi-modal prompt (handout, visible tests, buggy program, and execution feedback), candidate generation with test-based selection, and iterative feedback refinement — and drive it with the same model as CoEP, so every difference is attributable to the deployment paradigm, not the model. **PyDex+cache** re-prices the same queries under provider prompt caching (shared prefix written once at $1.25\times$, read at $0.1\times$) (Anthropic 2025). **CoEP** (ours) synthesizes one repairer per (assignment, model) once, then repairs locally: problem-specific rules and a shared generic edit library first, reference-guided minimal-hunk transplant for the structural remainder, and per-submission LLM fallback only for what abstains.

Because the cheapest tier of each provider is the natural home for a cost-driven method, we run every LLM arm at that tier: **GPT-5.4-nano** ($0.05/$0.40 per Mtok) and **Claude Haiku 4.5** ($1/$5 per Mtok). The non-LLM Refactory is model-independent. This yields the setting grid of Table 2.

Metrics

Repair rate (RR): percentage of buggy submissions for which the system produces a program passing the full test suite. **Relative patch size (RPS)**: tree-edit distance between buggy and repaired ASTs normalized by buggy program size, computed with the Refactory reference implementation (Hu et al. 2019); smaller patches preserve the student's approach and are pedagogically preferable, and RPS is *reported*, never used to gate a repair. **Average turnaround time (ATT)**:

mean wall-clock seconds to repair one submission end to end (local search plus any LLM-call latency). **Input/output tokens (InT/OuT)**: mean prompt and response tokens per repaired submission. **Cost**: USD per repaired submission and per assignment (including a system's one-off build cost), priced at the tier above from exact prompt/response token counts.

Three derived quantities tie the metrics to the claims. The **repair-source split** attributes every fix to the mechanism that produced it (synthesized rule, generic rule, reference transplant, or LLM fallback), grounding RQ3's attribution. The **abstention rate** f — the fraction of submissions the local tool escalates — is what the break-even $N^* = C_{\text{build}}/(\Delta(1-f))$ consumes. **Diagnosis coverage** is the fraction of repaired submissions whose patch carries a misconception label (rule-fired repairs), quantifying how much of the instructor-facing profile the tool actually delivers rather than leaving it implicit. Repair rates carry 95% Wilson intervals; paired comparisons use exact McNemar tests at the submission level, with per-assignment directions reported alongside pooled tests.

Protocol

No train/test split. Every arm sees the same data and is evaluated on *all* 2,970 buggy submissions. This is not an advantage granted to any one system but the shared regime of the setting: Refactory consumes all correct solutions before repairing, PyDex draws its in-context example fixes from the submission pool, and CoEP builds from a small sample of it. Since the baselines already exploit the whole pool of buggy and reference programs, CoEP using the same pool is the *identical* experiment, and building a repairer from programs drawn from the evaluation pool is temporally sound because the pool exists in full at feedback time. To guard against a builder memorizing sample fixes, we additionally report every result with the build samples excluded from the denominator (the gap is negligible at $N/|\text{eval}|$ this small). All LLM interactions — per-submission repairs and builder/refinement calls alike — are single-shot prompt→response exchanges with no code execution or test access at generation time; all validation is performed by our harness. Local search is deterministic; LLM calls are single-sample (n=1). Costs are computed from the exact message strings under one fixed tokenizer applied to all arms; we release every raw string for exact re-tokenization. Refactory ran in its original configuration ported to Python 3.11 on the same submissions; experiments ran on CPU only.

Results

Our evaluation answers three questions, one per claim the introduction makes. **RQ1 (Effectiveness & patch quality):** does the synthesized local tool repair as many submissions, with patches as small, as the hand-built symbolic SOTA and as per-submission LLM repair? **RQ2 (Cost structure):** what does each paradigm cost per submission and per assignment, and after how many submissions does CoEP's one-off build amortize? **RQ3 (Mechanism attribution):** which mechanism produces which repairs — LLM-authored rules, the generic library, reference transplant, or the fallback — is the LLM layer causally necessary (framework-only ablation), and how does builder capability across the two cheapest tiers change the tool?

Model	Method	RR (%)	RPS	ATT (s)	In/OuT	Cost ($)
non-LLM	Refactory	–	–	–	–	–
GPT-5.4 -nano	PyDex	–	–	–	–	–
	PyDex+cache	–	–	–	–	–
	CoEP	–	–	–	–	–
Claude Haiku 4.5	PyDex	–	–	–	–	–
	PyDex+cache	–	–	–	–	–
	CoEP	–	–	–	–	–

Table 2: Main comparison on 2,970 buggy submissions: repair rate, mean relative patch size over repaired submissions, average turnaround time, mean input/output tokens, and USD per repaired submission (with build cost amortized). Refactory is model-independent; PyDex, PyDex+cache, and CoEP are each run at the cheapest tier of both providers.

RQ1: Effectiveness and Patch Quality

Table 2 presents the headline comparison across the setting grid. *[Cells are populated from the full run over all 2,970 submissions.]* The central finding concerns CoEP's local repair against the strongest non-LLM tool. On the Refactory corpus (1,686 buggy submissions), CoEP's local path — named rules and the shared generic library, then reference-guided minimal-hunk transplant — repairs **97.8%** of submissions at mean **0.326** RPS, exceeding Refactory's own reported **90.8%** at **0.40** on *both* axes: higher coverage and smaller patches, at zero marginal LLM cost. Per assignment the local path reaches 100% on four of the five and exceeds Refactory's per-problem repair rate on every assignment *[per-assignment table from the final validated run]*. This is the result that makes a *local* tool worth synthesizing: it is not merely cheaper than a symbolic tool that consumes every correct solution, it repairs more, and more surgically.

Against the per-submission LLM baselines, functional repair is near-saturated on CS1-scale programs, so RR differences are small; the separation is in patch size and cost (RQ2). Because CoEP's local patches are test-validated before a student sees them, its only local failure mode is abstention ($\perp$), never a wrong repair; abstentions take the per-submission LLM path, so the hybrid never sacrifices coverage.

RQ1. On the Refactory corpus CoEP's local repair reaches 97.8% RR at 0.326 mean RPS, beating the classical non-LLM tool (90.8%, 0.40) on *both* coverage and patch size — and exceeding its repair rate on every assignment — at zero marginal LLM cost, while the hybrid escalates abstentions so coverage is never lost.

RQ2: Cost and Break-Even

The three systems differ in *cost structure*, not just constant. A per-submission LLM arm (PyDex) pays an inference on every submission, so its cost grows linearly with enrollment and

recurs every term; prompt caching (PyDex+cache) discounts the shared prefix but leaves the linear structure intact. CoEP pays a one-off synthesis cost C_{build} per assignment, then repairs the covered fraction at *zero* marginal LLM cost locally and pays a per-submission inference only on the abstaining tail.

Let Δ be the marginal per-submission saving of CoEP over a competitor and f the abstention (fallback) rate; synthesis wins once the enrollment exceeds $N^* = C_{\text{build}}/(\Delta(1-f))$. The expression is invariant to token price: C_{build} and Δ scale together, so a cheaper model or provider caching shrinks both sides equally and does not change N^* — the claim is about *how many model calls per specification* ($N \to 1$), not the price of a call, which is why running every arm at the cheapest tier does not blunt it. *[Table 2's Cost column and per-assignment N^* are populated from the full run.]* Synthesis is cheap: a repairer costs on the order of \$0.01–0.05 per assignment (e.g. \$0.04 with Claude Haiku 4.5, \$0.004 with GPT-5.4-nano on the Refactory assignments), the price of a handful of baseline repairs, paid once.

Beyond dollars, the covered fraction is repaired in about a second on a CPU with no network dependency, rate limits, or nondeterminism — properties a hosted per-query API does not offer, and that matter most at deadline-night submission spikes.

RQ2. CoEP replaces a per-submission inference with a one-off \$0.01–0.05 synthesis plus a fallback-only tail, breaking even at $N^* = C_{\text{build}}/(\Delta(1-f))$; the bound is price-invariant, so cheaper models and caching shrink both sides without changing when amortization wins.

RQ3: Mechanism Attribution and Builder Capability

Two complementary local mechanisms. CoEP's local repair combines named rules (problem-specific plus the shared generic library) with reference-guided transplant, and the two carry different loads. The rules make small, *labelled* edits — a relaxed comparison, an off-by-one bound, a wrong return — for the common misconceptions, at very small patch size (mean RPS ≈ 0.03). But most student bugs are not single-token slips; they need a medium, structural edit no token-level rule expresses, and there the reference-guided transplant supplies it by adopting the smallest set of differing hunks from a structurally-matched correct program, variable-renamed to fit. Across the Refactory corpus the rules fix about one repaired submission in seven ($\sim 13\%$) and the transplant the rest, so neither alone reaches the coverage of the pair; the transplant is what lifts a rule-only tool (which caps near 18%) to competitive coverage, and the rules are what keep the common cases at Refactory-beating patch sizes and provide the diagnosis below.

Diagnosis for free. Every rule-produced patch carries the ordered rule names that produced it, so aggregated over a class the tool yields a misconception profile — how often each named mistake occurred, on which programs — a report an instructor gets at no extra cost and a per-submission API never emits. This is a property of the artifact, not the coverage number: it is why a local, auditable repairer is worth synthesizing even where a hosted model would pass the same tests.

Builder capability. Synthesis is run at the cheapest tier of each provider (GPT-5.4-nano, Claude Haiku 4.5). Showing the builder *worked (buggy → fixed) pairs* rather than raw buggy samples turns "infer the fix, then generalize" into "generalize this shown repair," which the weak models do well (build-sample coverage rises from single digits to 75% at these tiers), and the shared generic library plus the reference engine make the local path robust to a weak builder — on the cheapest model the generic and reference mechanisms carry the majority of repairs. Capability is bought once per assignment, not once per submission; that is the entire point.

RQ3. Named rules (RPS ≈ 0.03, with a misconception label) cover the common slips; reference-guided transplant supplies the structural edits that lift coverage from a rule-only $\approx 18\%$ to Refactory-beating, and the rule provenance yields a per-class diagnosis a hosted API cannot.

Related Work

APR for programming assignments. The pre-LLM systems surveyed in the introduction (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022) exploit per-assignment structure but need many prior solutions; our transplant stage stands openly in their reference-reuse lineage, at hunk rather than block granularity, and our head-to-head run shows the composed artifact exceeding the strongest of them on both coverage and patch size. Closest to our synthesis layer, Refazer learns per-assignment syntactic transformations from (buggy, fixed) student pairs with PROSE (Rolim et al. 2017); CoEP replaces the example-driven synthesizer with a cheapest-tier LLM that *names* the misconception behind each rule, and surrounds the learned layer with generic search, reference transplant, and harness-gated escalation. At AAAI the setting stays active from the per-submission side: counterexample-guided and preference-tuned minimal repair at AAAI-25 (Orvalho, Janota, and Manquinho 2025; Dai et al. 2025), and AAAI-26's learner-tailored repair, which retrieves per-assignment example fixes to guide each query (Dai et al. 2026), amortizing *data* into a retrieval index while every repair still pays for an LLM call (our PyDex arm bounds this design's economics). CoEP instead removes the per-submission call for everything its artifact covers and pays only on escalation.

LLM-based repair. AlphaRepair recast repair as generation (Xia and Zhang 2022); ChatRepair adds conversational test feedback and per-bug cost accounting (Xia and Zhang 2024); PyDex/MMAPR target student programs (Zhang et al. 2024, 2022); FastFixer accelerates per-query inference (Liu et al. 2024); CREF adds a tutor in the loop (Yang et al. 2024). All are per-submission; CoEP is complementary (any can serve as its fallback) but shifts the primary cost to the assignment.

Templates, cost-efficient inference, and tool synthesis. PAR and TBar showed a fixed library of human-curated fix templates repairs a surprising share of real bugs (Kim et al. 2013; Liu et al. 2019); CoEP's generic layer is a mutation-

inversion instance (Just et al. 2014), and its novelty is the *synthesized, assignment-specific* template layer, authored per specification by an LLM. FrugalGPT cascades, SRepair's cheap pipelines, and provider caching (Chen, Zaharia, and Zou 2024; Xiang et al. 2024; Anthropic 2025; OpenAI 2024) all optimize *within* the pay-per-query paradigm and compose with CoEP, which instead removes queries. Voyager and LLMs-as-tool-makers amortize agent-written code across episodes (Wang et al. 2024; Cai et al. 2024), self-refinement improving one-shot generations (Madaan et al. 2023; Shinn et al. 2023); CoEP adds what education forces: verifiability (never-trust-self-reports) and a quality cap, because its output is feedback to a learner.

Ethical Statement

CoEP automates feedback, not grading, under instructor oversight; unhandled students get full-LLM treatment; repairs are auditable; data are public.

References

Ahmed, U. Z.; Fan, Z.; Yi, J.; Al-Bataineh, O. I.; and Roychoudhury, A. 2022. Verifix: Verified Repair of Programming Assignments. *ACM Transactions on Software Engineering and Methodology*, 31(4).

Anthropic. 2025. Prompt Caching — Claude API Documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching. Cache reads are billed at 0.1x the base input-token price; accessed July 2026.

Cai, T.; Wang, X.; Ma, T.; Chen, X.; and Zhou, D. 2024. Large Language Models as Tool Makers. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.

Chen, L.; Zaharia, M.; and Zou, J. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*.

Codeforces. ???? Codeforces Rating System. https://codeforces.com/blog/entry/68288. Rating bands: Newbie, Pupil, Apprentice, Specialist, Expert, ...

Dai, Z.; Chen, B.; Zhao, Z.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2025. Less is More: Adaptive Program Repair with Bug Localization and Preference Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Dai, Z.; Zhao, Z.; Wang, H.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 30467–30475.

Gulwani, S.; Radicek, I.; and Zuleger, F. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*, 465–480. ACM.

Hu, Y.; Ahmed, U. Z.; Mechtaev, S.; Leong, B.; and Roychoudhury, A. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE.

Just, R.; Jalali, D.; Inozemtseva, L.; Ernst, M. D.; Holmes, R.; and Fraser, G. 2014. Are Mutants a Valid Substitute for Real Faults in Software Testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2014*. ACM.

Kim, D.; Nam, J.; Song, J.; and Kim, S. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering, ICSE 2013*. IEEE.

Liu, F.; Liu, Z.; Zhao, Q.; Jiang, J.; Zhang, L.; Li, G.; Sun, Z.; Li, Z.; and Ma, Y. 2024. FastFixer: An Efficient and Effective Approach for Repairing Programming Assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024*. ACM.

Liu, K.; Koyuncu, A.; Kim, D.; and Bissyandé, T. F. 2019. TBar: Revisiting Template-Based Automated Program Repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019*, 31–42. ACM.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

OpenAI. 2024. Prompt Caching — OpenAI API Documentation. https://platform.openai.com/docs/guides/prompt-caching. Automatic prefix caching with discounted cached input tokens; see also https://openai.com/index/api-prompt-caching/; accessed July 2026.

Orvalho, P.; Janota, M.; and Manquinho, V. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Penedo, G.; Kydlíček, H.; Allal, L. B.; Lozhkov, A.; Bakouch, E.; von Werra, L.; and Wolf, T. 2025. Codeforces. https://huggingface.co/datasets/open-r1/codeforces.

Rolim, R.; Soares, G.; D'Antoni, L.; Polozov, O.; Gulwani, S.; Gheyi, R.; Suzuki, R.; and Hartmann, B. 2017. Learning Syntactic Program Transformations from Examples. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, 404–415.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*. ArXiv:2305.16291.

Wang, K.; Singh, R.; and Su, Z. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory

Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*. ACM.

Xia, C. S.; and Zhang, L. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*. ACM.

Xia, C. S.; and Zhang, L. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 819–831. ACM.

Xiang, J.; Xu, X.; Kong, F.; Wu, M.; Zhang, Z.; Zhang, H.; and Zhang, Y. 2024. How Far Can We Go with Practical Function-Level Program Repair? arXiv preprint arXiv:2404.12833.

Yang, B.; Tian, H.; Pian, W.; Yu, H.; Wang, H.; Klein, J.; Bissyandé, T. F.; and Jin, S. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 882–894. ACM.

Yi, J.; Ahmed, U. Z.; Karkare, A.; Tan, S. H.; and Roychoudhury, A. 2017. A Feasibility Study of Using Automated Program Repair for Introductory Programming Assignments. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017*, 740–751. ACM.

Zhang, J.; Cambronero, J.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2022. Repairing Bugs in Python Assignments Using Large Language Models. arXiv preprint arXiv:2209.14876. Preprint; the archival version of this work appeared as PyDex at OOPSLA 2024 (Zhang et al. 2024).

Zhang, J.; Cambronero, J. P.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2024. PyDex: Repairing Bugs in Introductory Python Assignments using LLMs. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1).

Reproducibility Checklist

General paper structure. Conceptual outline of the approach: yes (Figure 1 and the Approach section's loop specification). Claims vs. speculation delineated: yes (RQ answer boxes state measured claims; the closing recipe is marked as argument). Pedagogical references for background: yes (Related Work).

Theoretical contributions. This paper makes no formal theoretical claims; the break-even expression in RQ2 is elementary algebra over measured quantities, with all symbols defined in the text.

Datasets. All experiments use the publicly available Refactory dataset (Hu et al. 2019); our seeded, scripted import pipeline (test parsing, per-test entry points, split sampling) is released, so the exact evaluation sets are reconstructible from the public corpus. No new human-subject data are collected.

Computational experiments. Source code, prompts, synthesized repairers, raw LLM responses, the ported Refactory-tool configuration, and analysis scripts are released; experiments run on CPU only. Model versions, pricing snapshot, tokenizer convention, decoding budget (single sample), split seed, timeouts, and the RPS threshold are stated in Experimental Setup and fixed in the released configuration. Metrics are formally defined; repair rates carry 95% Wilson intervals and paired claims use exact McNemar tests. Local search and evaluation are deterministic; per-submission LLM calls are single-shot by design (deployment matches one attempt per submission). Builder synthesis is stochastic; every round's prompt, response, and build-set outcome for both builder tiers is released, and the refinement loop's convergence behavior is reported in RQ3.